1. JCL Modernization Executive Summary

1.1 Batch Job Overview

The batch execution sequence `CBEXPORT` (loaded from source file `CBEXPORT.jcl`) orchestrates mainframe batch workflows comprising 2 steps. It handles file preparation, program executions, database cursor refreshes, and resource routing.

1.2 Modernization Objectives

The primary migration goals include: - Decoupling step dependency chains into cloud-native scheduled workflows (e.g. Apache Airflow DAGs). - Replacing manual JCL dataset allocations with automated cloud storage operations. - Isolating custom programs from utility steps to translate and migrate them to modern application stacks.

1.3 Assumptions & Migration Scope

- **Assumptions**: Physical files reference cataloged storage partitions. Target environment runs comparable Python or Bash container executors.
- **Scope**: Includes step execution triggers, DD data mappings, utility steps replacements, and DAG orchestration structure mappings.

2. Step Execution Sequence & Classification

2.1 Step Execution Sequence

The following table inventories and categorizes the execution steps of the `CBEXPORT` job, providing clear target modernization recommendations:

Step Order	Step Label	Executed Program	Classification	Modernization Target / Rule
1	`STEP01`	`IDCAMS`	**Mainframe System Utility**	Replace with storage CLI or bucket lifecycle management tools
2	`STEP02`	`CBEXPORT`	**Custom Business Program**	Migrate source logic (e.g., translate COBOL/Assembler binary to Java/Python)

2.3 Library & Included Module Dependencies

The JCL batch stream defines search paths, load libraries, and include member structures as follows:

- No external JCLLIB library searches declared.

JOBLIB and STEPLIB Load Libraries

Partitioned Load Library: `AWS.M2.CARDDEMO.LOADLIB` (contains executed program binaries)

No include blocks or JCL member structures referenced.

3. Data & Resource Bindings

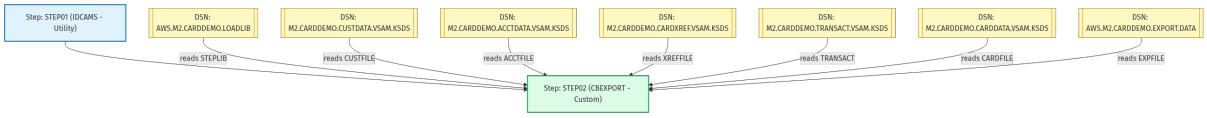
The physical files and datasets accessed by `CBEXPORT` steps are detailed below:

Step Label	DD Name	Dataset DSN	Access Pattern	Allocation / Disposition
`STEP01`	`SYSPRINT`	`SYSOUT`	System Print Log	`SHR`
`STEP01`	`SYSIN`	`SYSOUT`	System Print Log	`SHR`
`STEP02`	`STEPLIB`	`AWS.M2.CARDDEMO.LOADLIB`	Read (Input Store)	`SHR`
`STEP02`	`CUSTFILE`	`AWS.M2.CARDDEMO.CUSTDATA.VSAM.KSDS`	Read (Input Store)	`SHR`
`STEP02`	`ACCTFILE`	`AWS.M2.CARDDEMO.ACCTDATA.VSAM.KSDS`	Read (Input Store)	`SHR`
`STEP02`	`XREFFILE`	`AWS.M2.CARDDEMO.CARDXREF.VSAM.KSDS`	Read (Input Store)	`SHR`
`STEP02`	`TRANSACT`	`AWS.M2.CARDDEMO.TRANSACT.VSAM.KSDS`	Read (Input Store)	`SHR`
`STEP02`	`CARDFILE`	`AWS.M2.CARDDEMO.CARDDATA.VSAM.KSDS`	Read (Input Store)	`SHR`
`STEP02`	`EXPFILE`	`AWS.M2.CARDDEMO.EXPORT.DATA`	Read (Input Store)	`SHR`
`STEP02`	`SYSOUT`	`SYSOUT`	System Print Log	`SHR`
`STEP02`	`SYSPRINT`	`SYSOUT`	System Print Log	`SHR`

4. Program Structure & Execution Logic

4.1 JCL Flow Sequence Diagram

The diagram below outlines the sequential flow of execution steps and their corresponding dataset interactions:



4.2 JCL Step Structure & Parameters

The table below details the execution target, type classification, step-level conditions, and dataset resource mapping for each step:

Step Label	Executed Program	Classification	Step-Level COND	DD Configurations
`STEP01`	`IDCAMS`	System Utility	`None`	`SYSPRINT` -> `SYSOUT` (SHR) `SYSIN` -> `SYSOUT` (SHR)

STEP02	CBEXPORT	Custom Program	None	STEPLIB -> AWS.M2.CARDDEMO.LOADLIB (SHR) CUSTFILE -> AWS.M2.CARDDEMO.CUSTDATA.VSA (SHR) ACCTFILE -> AWS.M2.CARDDEMO.ACCTDATA.VSA (SHR) XREFFILE -> AWS.M2.CARDDEMO.CARDXREF.VSA (SHR) TRANSACT -> AWS.M2.CARDDEMO.TRANSACT.VSA (SHR) CARDFILE -> AWS.M2.CARDDEMO.CARDDATA.VSA (SHR) EXPFILE -> AWS.M2.CARDDEMO.EXPORT.DATA (SHR) SYSOUT -> SYSOUT (SHR) SYSPRINT -> SYSOUT (SHR)

4.3 Execution Logic and Control Flow

Modernizing mainframe job control logic requires translating traditional return/condition code checking to cloud orchestration logic:

Job-Level Conditional Check: None

Step-Level Condition Checks:

No step-level COND parameters defined.

Logical Block Control Structure:

- No structured IF-THEN-ELSE or ENDIF blocks parsed.

5. Modern Cloud Migration Blueprint

```
from airflow import DAG from airflow.operators.bash import BashOperator from
datetime import datetime, timedelta default_args = { 'owner': 'migration_team',
'start_date': datetime(2026, 1, 1), 'retries': 1, 'retry_delay':
timedelta(minutes=5), } with DAG( dag_id='cbexport_migration_dag',
default_args=default_args, schedule_interval=None, catchup=False,
description='Migrated orchestrations for JCL CBEXPORT', ) as dag: step01 =
BashOperator( task_id='step01', bash_command='python run_program.py --program
IDCAMS', dag=dag, ) step02 = BashOperator( task_id='step02',
bash_command='python run_program.py --program CBEXPORT', dag=dag, ) #
Dependencies step01 >> step02
```